

CSF (Cloth Simulation Filter) 算法流程分析报告

1. 算法概述

CSF 是一种基于布料模拟的机载 LiDAR 点云地面滤波算法，发表于 Remote Sensing 2016。核心思想：将一块虚拟布料从点云上方自由落下，布料与地形碰撞后形成的形状即为地面估计，通过计算原始点云到布料的距离来区分地面点与非地面点。

2. 完整算法流程

阶段一：点云输入与坐标变换

代码位置：CSF.cpp:52-99 (setPointCloud 系列方法)

- 输入原始点云 (X, Y, Z)
- 坐标系变换：y = -Z, z = Y，将 Y 轴翻转为布料下落方向（重力方向）
- 支持 OpenMP 并行拷贝 (#pragma omp parallel for)

阶段二：计算点云包围盒

代码位置：CSF.cpp:111-114 → point_cloud.cpp:21-40

- 计算点云的最小包围盒 bbMin 和 bbMax
- 遍历所有点，逐维度比较更新最小/最大值
- 用于确定布料的初始放置范围

阶段三：布料初始化

代码位置：CSF.cpp:116-147 → Cloth.cpp:22-95

1. 计算布料原点位置：

- `origin_pos = (bbMin.x - buffer*resolution, bbMax.y + 0.05, bbMin.z - buffer*resolution)`
- 布料放置在点云最高点上方，四周留2个粒子的缓冲区

2. 计算布料网格尺寸：

- `width_num = floor((bbMax.x - bbMin.x) / cloth_resolution) + 2 * buffer`
- `height_num = floor((bbMax.z - bbMin.z) / cloth_resolution) + 2 * buffer`

3. 创建粒子网格（Cloth.cpp:48-59）：

- 初始化 `width_num × height_num` 个粒子
- 所有粒子初始位置在同一水平面（`Y = origin_pos.y`）
- 每个粒子记录其在网格中的位置（`pos_x, pos_y`）

4. 建立约束关系（Cloth.cpp:62-94）：

- 近邻约束（距离1和 $\sqrt{2}$ ）：每个粒子与右、下、右下、左下邻居建立约束
- 次近邻约束（距离2和 $\sqrt{8}$ ）：每个粒子与距离2的邻居建立约束
- 约束通过 `neighborsList` 实现，不使用独立 `Constraint` 对象

阶段四：点云光栅化（Rasterization）

代码位置：Rasterization.cpp:102-146

1. 最近点投影（RasterTerrian 第106-130行）：

- 将每个 LiDAR 点投影到布料网格上最近的粒子
- 对每个粒子，记录最近的 LiDAR 点高度（`nearestPointHeight`）和索引
- 使用平方距离比较，避免开方运算

2. 空洞填充（RasterTerrian 第136-145行）：

- 对于没有对应 LiDAR 点的粒子（`nearestPointHeight == MIN_INF`）
- 使用扫描线法 `findHeightvalByScanline` 查找最近有效高度

- 扫描线方向：→右、←左、↑上、↓下
- 若扫描线失败，使用 BFS 邻居搜索 `findHeightValByNeighbor`

阶段五：布料物理模拟

代码位置： `CSF.cpp:152-168` → `Cloth.cpp:97-125`

1. 施加重力（`CSF.cpp:156`）：

- `gravity = 0.2`，方向为 `-Y`（向下）
- 力 = `(0, -0.2, 0) × time_step²`

2. 迭代模拟（默认 500 次迭代）：

每次迭代包含三个子步骤：

a) Verlet 积分更新位置（`Particle.cpp:24-30`）：

```
new_pos = pos + (pos - old_pos) × (1 - DAMPING) +
acceleration × time_step²
```

- `DAMPING = 0.01`，模拟阻尼
- 仅可移动粒子更新位置

b) 约束满足（`Particle.cpp:32-57`）：

- 每个粒子对其所有邻居执行约束校正
- 校正向量仅取 Y 分量： `correctionVector = (0, p2.y - p1.y, 0)`
- 根据刚性（`rigidness = constraint_iterations`）决定校正系数：
 - 双方可移动：使用 `doubleMove1` 系数表
 - 单方可移动：使用 `singleMove1` 系数表
 - `rigidness > 14` 时直接取 0.5 或 1.0

c) 地形碰撞检测（`Cloth.cpp:133-147`）：

- 若粒子 Y 坐标低于对应地形高度（`heightvals[i]`）
- 将粒子推回到地形表面
- 标记粒子为不可移动（`makeUnmovable`）

3. 早停机制（`CSF.cpp:163-166`）：

- 计算所有可移动粒子的最大位移 `maxDiff`
- 若 `maxDiff < 0.005`，提前终止迭代

阶段六：后处理（坡度平滑）

代码位置： `Cloth.cpp:149-371`

仅在 `params.bsloopSmooth = true` 时执行：

1. 连通分量分析（`movableFilter`，第149-254行）：

- 使用 BFS 遍历所有仍可移动的粒子
- 找到连通的可移动粒子区域
- 若连通区域粒子数 > 50（`MAX_PARTICLE_FOR_POSTPROCESSIN`），进入后处理

2. 边缘点检测（`findUnmovablePoint`，第256-335行）：

- 对连通区域中的每个粒子，检查其4邻域是否有不可移动粒子
- 若相邻不可移动粒子满足条件：
 - 高度差 < `smoothThreshold`（0.3）
 - 粒子高度与地形高度差 < `heightThreshold`（9999）
- 则将此粒子吸附到地形表面并标记为不可移动

3. 坡面传播（`handle_slop_connected`，第337-371行）：

- 从边缘点开始 BFS 传播
- 满足平滑条件的邻居粒子也被吸附到地形

阶段七：地面/非地面分类

代码位置： `c2cdist.cpp:22-61`

1. 双线性插值计算布面高度：

- 对每个 LiDAR 点，找到布料网格中对应的4个粒子
- 使用双线性插值计算该位置的布面高度 `fx`

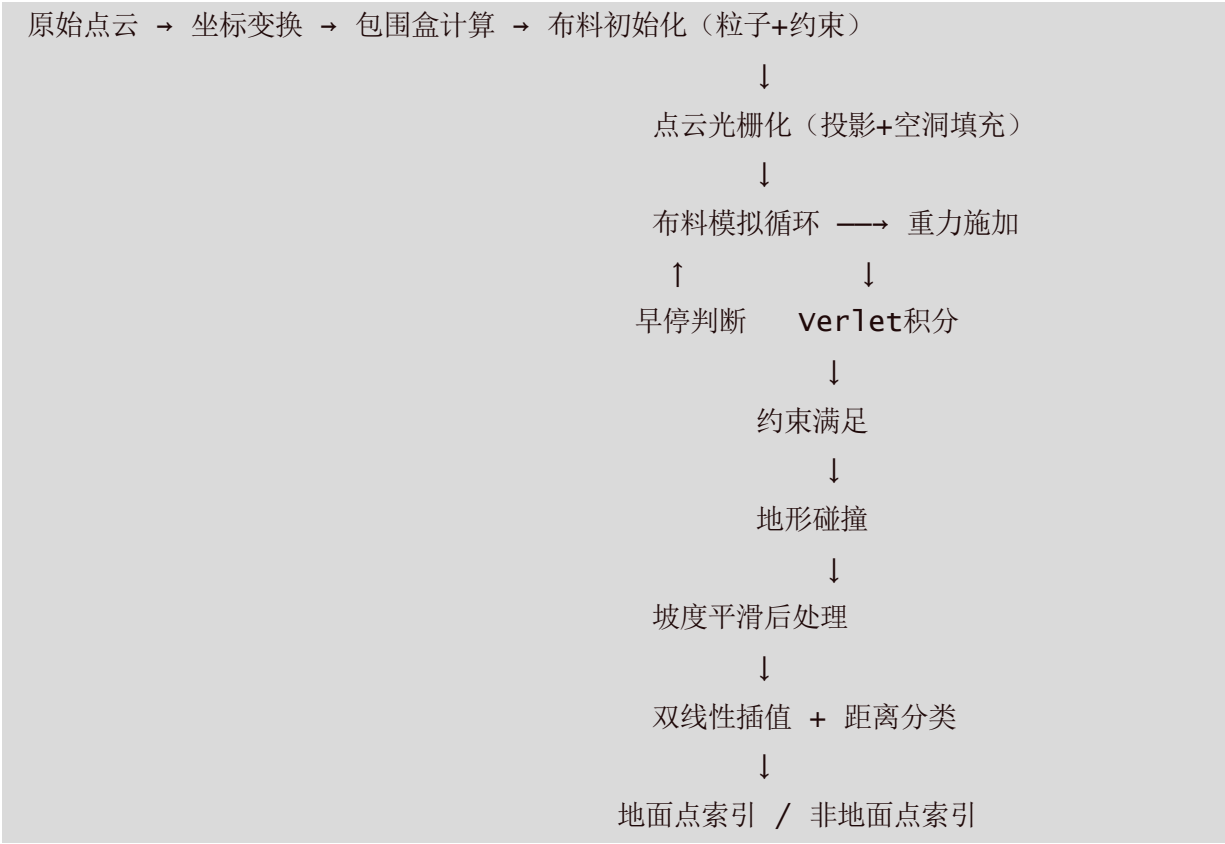
2. 距离判定：

- `height_var = fx - 点的y坐标`
 - 若 `|height_var| < class_threshold`（默认0.5）→ 地面点
 - 否则 → 非地面点
-

3. 算法参数汇总

参数	默认值	说明
cloth_resolution	1	布料网格分辨率（间距），越小精度越高
time_step	0.65	物理模拟时间步长
iterations	500	最大迭代次数
rigidness	3	刚性等级（1=软、2=中、3=硬），控制约束校正强度
class_threshold	0.5	地面分类距离阈值
bsloopSmooth	true	是否启用坡度平滑后处理

4. 关键数据流



5. 核心文件与职责

文件	职责
CSF.cpp/h	算法主流程编排、参数管理

文件	职责
<code>Cloth.cpp/h</code>	布料模拟核心：粒子管理、物理迭代、碰撞、后处理
<code>Particle.cpp/h</code>	粒子物理：Verlet积分、约束满足
<code>Rasterization.cpp/h</code>	点云到布料的光栅化投影、空洞填充
<code>c2cdist.cpp/h</code>	点云到布面距离计算与分类
<code>Constraint.cpp/h</code>	约束类（实际未被使用，被Particle内联替代）
<code>point_cloud.cpp/h</code>	点云数据结构与包围盒计算
<code>vec3.h</code>	三维向量运算
<code>XYZReader.cpp/h</code>	XYZ文件读取